MDQ Specification Audit: fill-in: mdq-spec

Date: 2026-10-20

Scope: The fill-in question type: `docs/question-types/fill-in.md` and `schema/fill-in.yaml`, read against `docs/references/grammar.md`, `docs/references/patterns.md`, `docs/question-types/question-base.md`, `docs/exam.md` and `docs/responses.md`. The blanks reuse the short-answer, numeric and choice rules; earlier audits already settled those rules.

Methodology

The audit script `scripts/spec_audit.py` passed all 15 steps before the audit started: it compiles every Lark snippet, checks the Additional Rules tables against `docs/lint-codes.md` and the corpus, and checks the frontmatter tables against the schemas.

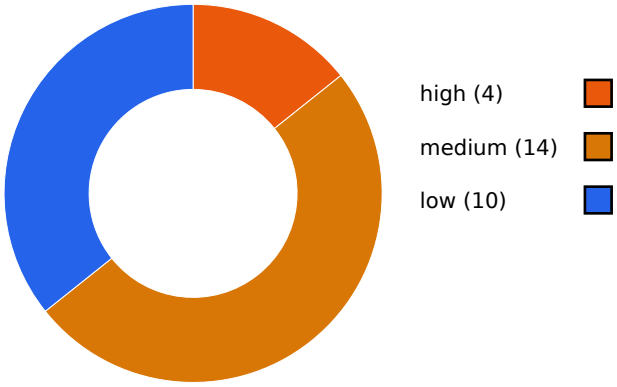
The lead auditor read `docs/question-types/fill-in.md` with the shared specs and recorded contradictions, ambiguities, omissions and inconsistencies. The audit focused on what is specific to fill-in: the blank references, the choice blanks, the frontmatter maps keyed by blank id, the grading strategies and the response shape. The blank rules that fill-in reuses from short-answer and numeric were checked for agreement with the decisions of the earlier audits.

One subagent turned the fill-in grammar and the shared grammar into Lark and tested what they accept and reject. A second, independent subagent scanned the fill-in document and schema, ran each complete example through `mdq validate`, and checked the Additional Rules table against the MUST, SHOULD and MAY requirements of the text.

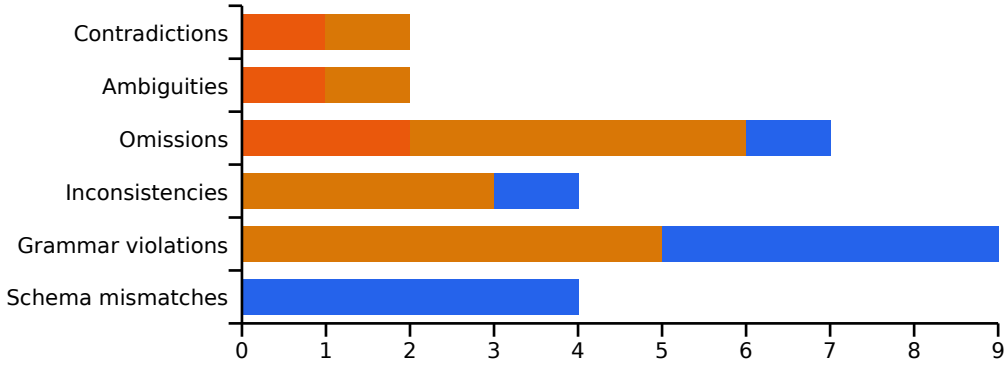
Executive summary

Severity	#
CRITICAL	0
HIGH	4
MEDIUM	14
LOW	10
INFO	0
Total findings	**28**

Findings by severity



Findings by category



Strengths and weaknesses

Strengths

✓ **Omissions:** The worked grading table covers all four strategies with empty, wrong, partial-credit and negative blank scores, and the values agree with the formulas. (docs/question-types/fill-in.md:436-457)

✓ **Inconsistencies:** The frontmatter maps (`preAccept`, `preReject`, `incorrectFeedback`, `domain`, `decimalPlaces`) each show the Markdown form and the JSON form, and the rules table gives the same two codes for each: `undefined-blank` for an unknown id and `schema-error` for a blank of the wrong kind. (docs/question-types/fill-in.md:51-132, 483-504)

✓ **Contradictions:** The numeric blank rules defer to the numeric question for domain, decimal places, exact arithmetic, units and valid responses, without restating them, so the two cannot drift. (docs/question-types/fill-in.md:255-260)

✓ **Schema mismatches:** The NumericBlank `answer` and `unit` patterns of schema/fill-in.yaml match the numeric grammar (exponent of at most 3 digits, nonzero denominator) and the UNIT terminal of grammar.md, and BlankId matches SLUG. (schema/fill-in.yaml:85-90, schema/fill-in.yaml:191-211)

✓ **Omissions:** Every complete Markdown example of fill-in.md (lines 8-19, 249-253, 267-278, 331-342) validates with mdq-py with no warning, and the arithmetic of the grading example table checks out row by row. (docs/question-types/fill-in.md:441-451)

✓ **Inconsistencies:** The frontmatter maps (preAccept, preReject, incorrectFeedback, domain, decimalPlaces) document their mapping onto blank fields with before/after YAML, and the corpus covers an undefined key and a key on the wrong kind of blank. (docs/question-types/fill-in.md:51-132)

✓ **Grammar violations:** `numeric_def` holds, and its two copies in fill-in.md are identical and match the numeric answer tag. Tested with Lark: accepted `[^size/numeric]: 2750000 +- 10%`, `[^size/numeric]:42`, `[^length/numeric(km)]: 6400 +- 5%`, `[^x/numeric(m/s)]: 1`, `-1/3`, `6.022e23 +- 0.001e23`, `3.14 +- 5% +- 0.1`, `3.14+-0.1`, `1e05`; rejected `1/0`, `1e3/2`, `1e1000`, `01`, `3,14`, `1 000`, `3.14 ± 0.1`, an empty body, `[^x/numeric (km)]`, `[^x/numeric()]`, `[^x/numeric/accept]`, `[^x/Numeric]`. (docs/question-types/fill-in.md:193, docs/question-types/fill-in.md:236, docs/question-types/numeric.md:75-86)

✓ **Grammar violations:** `accept_def` and `reject_def` hold, their two copies in fill-in.md are identical, and they match `accept_block`/`reject_block` of short-answer.md with the `[^SLUG` prefix. Tested with Lark: accepted single-line literal, exact (`` ` date ` ``), `regex (/a/b/, /[Bb]ras[íi]lia/i)`, `multi-line inexact, exact and regex content, lazy content after an empty tag, an empty accept tag, the /accept alias and list forms with feedback, comment and lazy lines; rejected an empty reject tag, [^planet/short-answer(km)], /short-answer/accept/reject, an unknown kind (/essay), /usr (unclosed regex), math.isnan (x) , a >` line after content or after an empty tag, a list after a blank line, and a list right after content. (docs/question-types/fill-in.md:195-196, docs/question-types/fill-in.md:289-292, docs/question-types/short-answer.md:148-166)

✓ **Grammar violations:** `choice_def` holds and its two copies are equivalent (`"[^" SLUG "]"` and `blank`). With list-item content as grammar.md defines it, Lark accepted the choice-blank-rich items (`[*] [andes] ...` with feedback and comment, `[50%]`, `[-25%]`), `[^capital]:` with trailing ws and up to 3 spaces of indentation; it rejected a blank line before the list, text after the colon followed by a list, and the multiple-selection value `[x]`. (docs/question-types/fill-in.md:192, docs/question-types/fill-in.md:219-220)

✓ **Grammar violations:** The stem rule accepts the stems of the corpus: two blanks in one sentence, a stem that starts with a blank, and blanks separated by plain text; it rejects a stem without a blank, in line with the `blanks` minimum of one. (docs/question-types/fill-in.md:141-142)

Central risks

• The stem of a fill-in question has no stated limits: which paragraphs form it, which blocks and inline elements may hold a blank, and which code each wrong place gives are settled only by the corpus and by the reference implementation.

• Several fill-in rules exist only in the corpus (parse errors of blank definitions) or only in prose with two allowed behaviors, so a second implementation that follows the text can disagree with mdq-py.

• No grading corpus exercises fill-in, so the four strategies, empty blanks and pending blanks can drift between mdq-py and mdq-js.

Detailed findings

Contradictions

Severity	Location	Description
HIGH	docs/question-types/fill-in.md:155-160	**F1** Prose allows two behaviors for a misplaced blank; the rules table and the corpus require specific codes
MEDIUM	docs/question-types/fill-in.md:438-439	**F22** The example table claims one set of blank scores for every strategy, but the wrong pick scores differ by strategy

F1: Prose allows two behaviors for a misplaced blank; the rules table and the corpus require specific codes

Location: docs/question-types/fill-in.md:155-160

```
illegal. A `[^slug]` found in any of them is not a blank: implementations MUST
either reject the document or leave the text exactly as written, and MUST NOT
silently drop it.
```

Description: The prose lets an implementation either reject a `[^slug]` in a heading, list item, table cell, blockquote or fenced code, or keep it as text. The Additional Rules table has one rule, `misplaced-blank` (error) for 'blanks may only appear in a plain text run of a paragraph'. The corpus uses three different outcomes: a heading gives `unreferenced-blank` (`examples/invalid/fill-in-misplaced-blank-heading`), a code span gives `misplaced-blank` (`examples/invalid/fill-in-misplaced-blank-code-span`), and fenced code is valid text (`examples/valid/fill-in/blank-in-code-is-text`). Nothing in the spec says which block kind gives which outcome. Lines 174-176 also put inline code with link text and emphasis, but a code span is literal text in CommonMark, as fenced code is.

Impact: Two conforming implementations can give different lint codes, or one accepts a document the other rejects. The corpus fixes behavior that the spec does not state.

Suggested fix: Give one outcome per context in a table: for example, fenced code, indented code and code spans are literal text; a `[^slug]` in a heading, list item, table cell or blockquote of the stem, or inside emphasis or link text, is `misplaced-blank`. Remove 'either ... or'.

Exploitability conditions: A `[^slug]` in a heading, list item, table cell, blockquote, code span or emphasis of the stem.

F22: The example table claims one set of blank scores for every strategy, but the wrong pick scores differ by strategy

Location: docs/question-types/fill-in.md:438-439

```
The "blank scores" column lists what each blank scored on its own, in that
order, and every strategy is computed from those three numbers.
```

Description: The column shows `-0.5` for a wrong pick, which line 453-456 says holds only under `symmetric`; under the other three strategies the same pick scores 0. So the other columns are not computed from those three numbers. The `partial` values match only because `partial` also clamps negatives to 0.

Impact: Readers can conclude that a choice blank scores -0.5 under `partial` and that the clamp in `partial` is what removes it.

Suggested fix: Say that the column holds the `symmetric` blank scores and that the wrong pick scores 0 under the other strategies, or show both values.

Exploitability conditions: None.

Ambiguities

Severity	Location	Description
HIGH	docs/question-types/fill-in.md:407-410	**F4** Under no-mistakes and all-or-nothing, the multiple choice strategy for a choice blank is not stated
MEDIUM	docs/question-types/fill-in.md:357-360	**F11** For a pending fill-in, the spec does not say what score the instructor supplies

F4: Under no-mistakes and all-or-nothing, the multiple choice strategy for a choice blank is not stated

Location: docs/question-types/fill-in.md:407-410

```
* A **choice blank** is graded like a
  [multiple choice](multiple-choice.md#grading) question, under the strategy
  the fill-in question declares.
```

Description: Multiple choice accepts only `partial` and `symmetric` (`docs/exam.md:357-358`). The fill-in question can declare `no-mistakes` or `all-or-nothing`, which a multiple choice question does not accept, so 'under the strategy the fill-in question declares' has no meaning for these two. Lines 453-456 imply the answer only through an example ('Under `partial`, `no-mistakes` and `all-or-nothing` that same wrong pick scores 0').

Impact: An implementation can raise an error, or score the blank with `symmetric`, for a fill-in under `no-mistakes`.

Suggested fix: Add a mapping: a choice blank is scored with multiple choice `partial` under `partial`, `no-mistakes` and `all-or-nothing`, and with `symmetric` under `symmetric`, as in the exam table at `docs/exam.md:346-351`.

F11: For a pending fill-in, the spec does not say what score the instructor supplies

Location: docs/question-types/fill-in.md:357-360

```
A response to
the question is settled only when the response to every blank is settled; if
a blank leaves its response to the instructor, the whole question is pending.
```

Description: `docs/responses.md` (Pending) says a pending response has no score until the instructor supplies one. For a fill-in, the instructor can score the pending blank (then the strategy combines it with the other blanks) or the whole question (then the strategy is bypassed). The range of a blank score from the instructor (only 0 or 1, as in a short answer blank, or any value in [0, 1]) is also not stated.

Impact: An LMS cannot tell which value to ask the instructor for, and `no-mistakes` gives different results for a blank score of 0.5.

Suggested fix: State that the instructor scores each pending blank, in [0, 1], and that the strategy then combines the blank scores as usual.

Omissions

Severity	Location	Description
HIGH	docs/question-types/fill-in.md:137-148	**F2** The spec does not say which paragraphs form the stem, so blanks in two paragraphs fail
HIGH	docs/question-types/fill-in.md:403	**F5** An empty blank in the response is not defined
MEDIUM	docs/question-types/fill-in.md:208-210	**F3** The spec does not say whether blank definitions must be contiguous
MEDIUM	docs/question-types/fill-in.md:185-187	**F6** A blank referenced more than once in the stem has no rule
MEDIUM	docs/question-types/fill-in.md:481-482, docs/question-types/fill-in.md:206, docs/question-types/fill-in.md:226, docs/question-types/fill-in.md:302-304	**F7** Several Markdown parse errors of blank definitions are only in the corpus
MEDIUM	docs/question-types/fill-in.md:389	**F23** No grading corpus for fill-in
LOW	docs/question-types/fill-in.md:155-157	**F26** No example exercises blanks in list items, table cells, blockquotes or a preamble paragraph

F2: The spec does not say which paragraphs form the stem, so blanks in two paragraphs fail

Location: docs/question-types/fill-in.md:137-148

```
stem  : inline_md? (blank inline_md?)+
blank : "[^" SLUG "]"
```

Description: The stem grammar describes one run of inline content. The text never says where the preamble ends and the stem begins, or whether the stem can have more than one paragraph with blanks. `docs/question-types/question-base.md:132-134` only says implementations SHOULD require a one-paragraph stem. With the reference implementation, the document 'Intro.\n\nFirst [^a] paragraph.\n\nMiddle.\n\nSecond [^b] paragraph.\n\n[^a/short-answer]: x\n\n[^b/short-answer]: y' fails with `unreferenced-blank` for a: the first paragraph with a blank goes to the preamble, where line 178 says a `[^slug]` is literal text. The spec does not exercise this case and does not say it is an error.

Impact: Authors write cloze texts that span paragraphs. The result is an error that names the wrong cause, and another implementation can accept the same document.

Suggested fix: State the rule: either the stem is the last paragraph before the definitions (and a `[^slug]` in an earlier paragraph gives a specific error), or the stem is every block from the first paragraph with a blank to the definitions. Add an example to the corpus.

Exploitability conditions: Blanks in two or more paragraphs.

F5: An empty blank in the response is not defined

Location: docs/question-types/fill-in.md:403

```
A blank the student left empty scores 0 under every strategy.
```

Description: `docs/responses.md:213-231` does not say how a response marks an empty blank: a missing key, a `null` value, or both. For a short answer blank, "" is not empty: `docs/responses.md:200-204` says it is graded like any other text. The spec also does not say what an empty mapping {} is (all blanks empty, or a skip like `null`), nor whether a value of the wrong type for a blank (a number for a choice blank, a list for a short answer blank) is a `ResponseError`.

Impact: The strategies `no-mistakes` and `all-or-nothing` treat an empty blank differently from a wrong one, so the score depends on a rule the spec does not give.

Suggested fix: In `docs/responses.md`, Fill in the blanks: a missing key and a `null` value are an empty blank; {} is a response with every blank empty (the question is not skipped); a value of the wrong type is a malformed response by the rule of the blank's type. Add response cases to the grading corpus.

F3: The spec does not say whether blank definitions must be contiguous

Location: docs/question-types/fill-in.md:208-210

```
Definitions may appear in any order, and that order need not match the order
the stem references them.
```

Description: The text allows any order but does not say if a paragraph can stand between two definitions. In the reference implementation, a paragraph after the first definition ends the body: a later `[^b/short-answer]: y` goes to the epilogue as text, and the stem reference `[^b]` gives `undefined-blank`, which names the wrong cause.

Impact: An author who puts an explanation between definitions gets an error that points at the stem, not at the definition.

Suggested fix: State that the definitions form one sequence of blocks, separated only by blank lines, and that the first other block starts the epilogue. Consider a dedicated lint for a definition tag in the epilogue.

Exploitability conditions: A paragraph between two blank definitions.

F6: A blank referenced more than once in the stem has no rule

Location: docs/question-types/fill-in.md:185-187

```
Every blank referenced by the stem MUST be defined, and every definition MUST
be referenced by the stem.
```

Description: Nothing says whether the stem can reference the same id twice, as in 'The capital is [^c]. Write [^c] again.'. The response has one value per id, so two inputs would share one answer.

Impact: An implementation can render two inputs bound to one key, render one, or reject the document.

Suggested fix: State that each blank id appears at most once in the stem, with an error code (for example `duplicate-blank-reference`), or state that repeated references share one input.

F7: Several Markdown parse errors of blank definitions are only in the corpus

Location: docs/question-types/fill-in.md:481-482, docs/question-types/fill-in.md:206, docs/question-types/fill-in.md:226, docs/question-types/fill-in.md:302-304

```
| blanks[]             | error    | parse-error        | a short answer blank defines each
block at most once[^5]           |
```

Description: The corpus gives `parse-error` for cases the Additional Rules table does not list and the text does not settle: a blank line between `[^bioma]:` and its choice list (`examples/invalid/fill-in-blank-line-before-choices`; line 226 says 'must start on the following line' but gives no code), an unknown kind suffix such as `[^resposta/essay]:` (`fill-in-unknown-blank-kind`), a unit on a short answer tag (`fill-in-unit-on-short-answer-tag`; line 206 only says 'not valid syntax'), two choice definitions for one id (`fill-in-two-choice-definitions`) and one id with two kinds (`fill-in-two-kinds-one-blank`). Text on the tag line of a choice definition (`[^capital]: Brasília`) has no stated outcome. The table also omits the rule of line 319-321 that a list item right after the content of a short answer definition is a `parse-error`; the short answer table has it (`docs/question-types/short-answer.md:511`).

Impact: An implementation that reads only the spec can accept these documents or report other codes.

Suggested fix: State each rule in the text with MUST and add a `parse-error` row per case: blank line before the choice list, text on a choice tag line, unknown kind suffix, unit on a non-numeric tag, a second definition of an id (any kind), and a list item right after short answer content.

F23: No grading corpus for fill-in

Location: docs/question-types/fill-in.md:389

```
## Grading
```

Description: examples/grading has corpora for multiple choice, multiple selection, true/false, numeric and short answer, but none for fill-in. No shared test exercises the four combination strategies, the example table, empty blanks, a pending short answer blank making the whole question pending, the feedback order of the flat list (line 384-386), or the automation of a mixed question.

Impact: mdq-py and mdq-js can diverge on fill-in scores and feedback without any shared test failing.

Suggested fix: Add examples/grading/fill-in.yaml covering the table rows, empty blanks, pending blanks and feedback order.

Exploitability conditions: None.

F26: No example exercises blanks in list items, table cells, blockquotes or a preamble paragraph

Location: docs/question-types/fill-in.md:155-157

```
At the block level, the hosting block MUST be an ordinary paragraph. Blanks in
```

Description: The corpus covers a heading, a link, emphasis, a code span, fenced code and an epilogue paragraph. It has no case for a `[^slug]` in a list item, a table cell, a blockquote (blockquote-preamble has none inside the quote), or an ordinary preamble paragraph (line 178-180).

Impact: These MUST rules are not checked by the shared tests.

Suggested fix: Add valid or invalid examples for each block kind once the behavior is settled.

Exploitability conditions: None.

Inconsistencies

Severity	Location	Description
MEDIUM	docs/question-types/fill-in.md:306-308	**F17** A short answer blank with only a reject block is not addressed, unlike a short answer question that MUST have an accept block
MEDIUM	docs/question-types/fill-in.md:314-315	**F18** A blank with an empty accept block does not always leave its responses to the instructor
MEDIUM	docs/question-types/fill-in.md:197	**F19** The two grammar snippets of `patterns` disagree
LOW	docs/question-types/fill-in.md:213-215	**F10** A choice blank has three names: choice blank, multiple choice blank and type multiple-choice

F17: A short answer blank with only a reject block is not addressed, unlike a short answer question that MUST have an accept block

Location: docs/question-types/fill-in.md:306-308

```
A short answer blank has the two blocks of a short answer question:
```

Description: short-answer.md line 140 says a question MUST have a `[short-answer]` block. The fill-in grammar `short_answer_def : accept_def | reject_def` lets a blank consist of only `[^b/short-answer/reject]:`, and mdq-py accepts it. The fill-in text says the blank has the blocks of a short answer question and that it carries the rules of short answer (line 465-469), which suggests the accept block is required. No example covers a reject-only blank.

Impact: One implementation may require the accept block by inheritance, another may not.

Suggested fix: State whether the accept block is required for a blank, and add an example either way.

Exploitability conditions: A short answer blank defined only by a reject block.

F18: A blank with an empty accept block does not always leave its responses to the instructor

Location: docs/question-types/fill-in.md:314-315

```
`[^slug/short-answer]` block has no pattern has no `accept` list, and the
instructor grades its responses, see [Automation](#automation).
```

Description: By short-answer.md (Grading, Automation), a blank with no accept pattern but with a reject block is semi-automatic: the reject patterns settle some responses. With `unmatched: incorrect` in the frontmatter, no response goes to the instructor (and `no-correct-answer` is reported). The sentence says the instructor grades its responses without these conditions.

Impact: Readers may think reject patterns or `unmatched` have no effect on an empty accept block.

Suggested fix: Say that the blank has no accept list and that `unmatched` defaults to `manual`, so the instructor grades the responses that no reject pattern settles.

Exploitability conditions: An empty `[^slug/short-answer]:` with a reject block or with `unmatched: incorrect`.

F19: The two grammar snippets of `patterns` disagree

Location: docs/question-types/fill-in.md:197

Mechanical fix

```
patterns        : answer | nl pattern_list
```

Description: The Blank definitions snippet defines `patterns : answer | nl pattern_list`, while the Short answer blanks snippet (line 291) and short-answer.md define `patterns : content | nl lazy_content | nl pattern_list`. `answer` is not defined here and is the name of the tag rule in numeric.md; the lazy-content alternative after an empty tag is missing.

Impact: The first snippet rejects valid documents (content on lazy lines after an empty tag) and refers to an unrelated rule.

Suggested fix: Replace line 197 with `patterns : content | nl lazy_content | nl pattern_list`, as on line 291.

F10: A choice blank has three names: choice blank, multiple choice blank and type multiple-choice

Location: docs/question-types/fill-in.md:213-215

Mechanical fix

```
## Multiple choice blanks

Multiple-choice blanks are represented by a reference definition followed by a
```

Description: The rules table, the Feedback and the Grading sections say 'choice blank'; the section heading and its anchor say 'Multiple choice blanks'; the JSON `type` is `multiple-choice`. The other two kinds use one term each (numeric blank, short answer blank).

Impact: Minor: readers must map three terms to one concept.

Suggested fix: Use 'choice blank' in prose and headings, and state once that its JSON `type` is `multiple-choice`.

Grammar violations

Severity	Location	Description
MEDIUM	docs/question-types/fill-in.md:141	**F32** Stem rule allows only one inline element between blanks
MEDIUM	docs/question-types/fill-in.md:142	**F33** `blank` matches the text of a link `[^x](url)`
MEDIUM	docs/question-types/fill-in.md:168-172	**F34** Split reading of `**start [^blank] end**` is legal under the stem rule
MEDIUM	docs/references/patterns.md:252	**F37** `pattern_item` includes the list marker, `item` does not
MEDIUM	docs/references/grammar.md:156	**F40** LAZY_START ignores setext heading underlines
LOW	docs/question-types/fill-in.md:223-224, docs/question-types/fill-in.md:239-241	**F38** Two `value` rules collide in a fill-in grammar
LOW	docs/references/patterns.md:252-254, docs/question-types/fill-in.md:292	**F39** `pattern_list` accepts a change of bullet character
LOW	docs/references/grammar.md:156, docs/question-types/short-answer.md:156	**F41** LAZY_START rejects lines that CommonMark reads as paragraph continuation
LOW	docs/question-types/short-answer.md:152	**F42** `exact` rejects a code span that opens at the end of a line

F32: Stem rule allows only one inline element between blanks

Location: docs/question-types/fill-in.md:141

Mechanical fix

```
stem  : inline_md? (blank inline_md?)+
```

Description: `inline_md` is defined as 'any valid inline markdown element' (singular, fill-in.md:145), and the rule allows at most one of them before, between and after blanks. Counter-example: the stem `The **capital** of Brazil is [^capital].` has three inline elements before the blank (text, strong, text). The text at fill-in.md:152-176 allows markup elsewhere in the paragraph as long as the blank itself is in a plain-text run, so this stem is valid, but the rule rejects it (tested with Lark, `inline_md` stubbed as TEXT | STRONG | LINK).

Impact: The grammar rejects ordinary stems with any formatting, or the reader must guess that `inline_md` means a sequence.

Suggested fix: Use `inline_md*` in the rule, or say that `inline_md` is a sequence of inline elements.

F33: `blank` matches the text of a link `[^x](url)`

Location: docs/question-types/fill-in.md:142

```
blank : "[^" SLUG "]"
```

Description: `CHOICE_ID`, `CHOICE_MARK` and `VALUE_END` exclude a `]` followed by `(` or `[` because that bracket starts a markdown link. `blank` has no such lookahead. Counter-example: in the stem `Veja [^nota](https://www.ibge.gov.br) agora.`, CommonMark (and markdown-it, where the footnote reference rule runs after `link`) reads a link with the text `^nota`. The stem rule reads the blank `nota` followed by the text `(https://www.ibge.gov.br) agora.` (tested with Lark: the only parse is blank + text). The same holds for `[^nota][ref]`.

Impact: The grammar and the markdown parser disagree on whether the stem has a blank, so an implementation that follows the grammar reports `undefined-blank` for a valid link.

Suggested fix: Add the lookahead of `CHOICE_ID` to the rule (`"]"` not followed by `(` or `[`), and say in the text that `[^x](url)` and `[^x][ref]` are links.

F34: Split reading of `**start [^blank] end**` is legal under the stem rule

Location: docs/question-types/fill-in.md:168-172

```
interpreted as an unclosed bold element `**start `, followed by a blank and then
the text ` end**`. In both approaches, the result would be illegal according to
the spec, so both parsing strategies are valid.
```

Description: The text says that splitting the paragraph around the blank also gives an illegal stem. It does not: CommonMark reads `**start ` alone as the literal text `**start ` (an unmatched delimiter run is text), so the split gives text, blank, text, which `stem : inline_md? (blank inline_md?)+` accepts. Only the whole-paragraph reading (a strong element that contains a blank) is illegal. The corpus (examples/invalid/fill-in-misplaced-blank-emphasis) expects `misplaced-blank`, so an implementation that uses the split strategy, which the text allows, accepts a document the corpus rejects.

Impact: The text allows a parsing strategy that gives a different result from the corpus.

Suggested fix: Require the markdown inline parse of the whole paragraph first, and state that a blank inside any inline element is `misplaced-blank`. Remove the claim that both strategies agree.

F37: `pattern_item` includes the list marker, `item` does not

Location: docs/references/patterns.md:252

```
pattern_item : ws? LIST_MARK ws content nl (blank_line* observations)?
```

Description: grammar.md:73-75 says the grammar of a list item applies to the content of the item after the removal of the list marker and its padding. The choice `item` of choices.md:66 follows that (it starts with `ws? "["`), but `pattern_item` starts with `ws? LIST_MARK ws`. fill-in.md uses both in the same way (`choice_list : item+`, `pattern_list : pattern_item+`, each 'the unordered list those items form'). Counter-example in Lark: the fill-in example `[^capital]: + * [ ] Lisbon + * [*] Brasília` is rejected by `choice_list : item+`, while `[ ] Lisbon` without `*` is accepted; for `pattern_list` it is the reverse. Applied to the content of an item as grammar.md says, `pattern_item` rejects `Jupiter` (the content of `* Jupiter`).

Impact: The two list rules that fill-in reuses follow opposite conventions; a literal reading of either one rejects the examples of the other.

Suggested fix: Remove `ws? LIST_MARK ws` from `pattern_item` (the list marker belongs to the list, as for `item`), or add it to `item` and change grammar.md:73-75. Pick one convention for both.

F40: LAZY_START ignores setext heading underlines

Location: docs/references/grammar.md:156

```
LAZY_START    : /(?!>|!(?!\[)|[*+-][ \t]|1[.)][ \t]|#{1,6}([ \t]|(?=[\r\n]))|`{3}|~{3}|(\*[
\t]*){3,}(?=[\r\n])|(-[ \t]*){3,}(?=[\r\n])|(_[ \t]*){3,}(?=[\r\n]))[^ \t\r\n]/
```

Description: One-line content follows the CommonMark paragraph, and a `---` or `===` line (and also a single `-` or `=`) right after a paragraph line is a setext heading underline: the whole paragraph, tag line included, becomes a heading. The terminal reads them differently. Counter-examples (tested with Lark on the fill-in `accept_def`): `[^p/short-answer]: Rio + ===` is accepted as the pattern `Rio ===`; `[^p/short-answer]: Rio + -` is accepted as `Rio -`; `[^p/short-answer]: Rio + ---` ends the block with a 'thematic break' (grammar.md:163, short-answer.md:284). In CommonMark all three turn the tag line into a heading, so there is no definition. exam.md:146 already describes this setext behavior for `---`.

Impact: An implementation that follows the grammar and one that follows the markdown AST disagree on whether the definition exists.

Suggested fix: Add the setext underline (=+ or -+ with optional trailing ws) to the exclusions of LAZY_START, and say what happens to a tag line that becomes a setext heading (for example a `parse-error`).

F38: Two `value` rules collide in a fill-in grammar

Location: docs/question-types/fill-in.md:223-224, docs/question-types/fill-in.md:239-241

```
with the `value` rule of [multiple choice](multiple-choice.md#body), and
```

Description: A choice blank uses the `item` rule with the `value` of multiple-choice.md:46 (`ws | ws? "*" ws? | ws? percent ws?`), and a numeric blank uses `numeric_body`, which uses the `value` of numeric.md:79 (`number | fraction`). A grammar of fill-in built from the referenced rules has two different rules called `value`. To compile the test grammar, the numeric one had to be renamed.

Impact: The fill-in grammar cannot be assembled from the referenced snippets without renaming a rule.

Suggested fix: Rename one of the rules, for example `numeric_value` in numeric.md or `choice_value` in the choice types.

F39: `pattern_list` accepts a change of bullet character

Location: docs/references/patterns.md:252-254, docs/question-types/fill-in.md:292

```
LIST_MARK    : /[*+-]/
```

Description: LIST_MARK is chosen per item, so `pattern_list : pattern_item+` accepts `* Jupiter` followed by `- Saturn` as one list (tested with Lark: accepted). In CommonMark a change of bullet character starts a new list, so the second item is a second list that follows the first without a blank line. The text says `pattern_list` is 'the unordered list those items form' (fill-in.md:299). The same holds for `choice_list : item+` (fill-in.md:220).

Impact: Grammar and markdown parse disagree on how many lists the block has; the second list may become epilogue content in an implementation that follows the markdown AST.

Suggested fix: State that all items use the same bullet character, or say that the grammar reads consecutive lists with different bullets as one list.

F41: LAZY_START rejects lines that CommonMark reads as paragraph continuation

Location: docs/references/grammar.md:156, docs/question-types/short-answer.md:156

```
lazy_content : ws? LAZY_TEXT (nl ws? LAZY_TEXT)*
```

Description: Counter-examples, each after `[^p/short-answer]: Rio` (tested with Lark on `accept_def`, all rejected): (1) `> Negro` and a tab followed by `! Negro`: with four columns of indentation the line cannot start a block quote, so CommonMark reads it as paragraph continuation; the grammar excludes `>` after any `ws?` and the text makes it a `parse-error`. (2) `*` and `1.` (empty list items): an empty list item cannot interrupt a paragraph, so CommonMark continues the paragraph; LAZY_START excludes `[*+-][ \t]` and `1[.)][ \t]` even when only ws follows. (3) A line that starts with a backtick fence whose info string has a backtick, such as `` `b ``, is not a fenced code block in CommonMark but LAZY_START excludes it.

Impact: Contrived inputs parse differently in the grammar and in CommonMark.

Suggested fix: Limit the `>`/`!` exclusion to at most three columns of indentation, exclude list markers only when non-blank text follows, and exclude a backtick fence only when its info string has no backtick.

F42: `exact` rejects a code span that opens at the end of a line

Location: docs/question-types/short-answer.md:152

```
exact        : FENCE EXACT_TEXT (nl ws? LAZY_EXACT_TEXT)* FENCE ws?
```

Description: The exact literal follows the CommonMark code span, where a line ending can follow the opening run. EXACT_TEXT is `+`, so the first line must have text after the fence. Counter-example: `[^p/short-answer]:` followed by a single backtick, then the line `Rio Negro` followed by a backtick. The lines join to a backtick, `Rio Negro` and a backtick: a complete exact literal by the text of short-answer.md:235-239, but the grammar rejects it (tested with Lark). A `schema-error` would be reported for a document that the text accepts.

Impact: Minimal; authors rarely break a literal right after the fence.

Suggested fix: Make the text on the first line optional (EXACT_TEXT?), or state that the opening run must be followed by text on the same line.

Schema mismatches

Severity	Location	Description
LOW	schema/fill-in.yaml:76-79	**F8** The Blank schema says a bare tag infers its kind from what follows
LOW	schema/fill-in.yaml:125-128, schema/fill-in.yaml:139-141	**F9** The ShortAnswerBlank description lists answer forms that no longer exist

Severity	Location	Description
LOW	schema/fill-in.yaml:38-39	**F29** `grading` description omits the per-type entry of an exam grading map
LOW	schema/fill-in.yaml:56-58	**F30** `unmatched` description drops the per-blank default

F8: The Blank schema says a bare tag infers its kind from what follows

Location: schema/fill-in.yaml:76-79

Mechanical fix

```
required here even though the surface syntax may leave it implicit
(`[^capital]:` infers its kind from the definition that follows,
while `[^size/numeric]:` states it).
```

Description: `docs/question-types/fill-in.md:203` says a bare `[^slug]:` is always a choice blank. The kind is not inferred from the definition.

Impact: A reader of the schema can expect `[^capital]: Brasília` to be a short answer blank.

Suggested fix: Write: '`[^capital]:` is a choice blank, while `[^size/numeric]:` and `[^city/short-answer]:` state their kind.'

F9: The ShortAnswerBlank description lists answer forms that no longer exist

Location: schema/fill-in.yaml:125-128, schema/fill-in.yaml:139-141

Mechanical fix

```
exactly like a short-answer question. Carries the same answer
machinery: a list of accepted responses, a regular expression, or
`accept`/`reject` pattern lists that can attach feedback.
```

Description: A short answer blank has only the `accept`/`reject` pattern lists (and `preAccept`/`preReject`); a regex is one form of pattern, not a separate field. The description of `accept` (line 140) also cites only the alias `[^id/short-answer/accept]`, not the main tag `[^id/short-answer]`.

Impact: Misleads readers of the schema about the fields of the blank.

Suggested fix: Write: 'Carries the same answer machinery: `accept` and `reject` pattern lists, which can attach feedback.' In `accept`, cite `[^id/short-answer]` (alias `[^id/short-answer/accept]`).

F29: `grading` description omits the per-type entry of an exam grading map

Location: schema/fill-in.yaml:38-39

Mechanical fix

```
`inherit`, the default, takes the exam's `grading` value, and
```

Description: fill-in.md line 398-399 says `inherit` takes the exam's `grading`, or its `fill-in` entry when the exam's `grading` is a map, through the exam translation table.

Impact: Minor.

Suggested fix: Mention the `fill-in` entry of a grading map.

F30: `unmatched` description drops the per-blank default

Location: schema/fill-in.yaml:56-58

Mechanical fix

```
Applies to every short answer blank of the question, with the
```

Description: The local description replaces the one of short-answer.yaml#/$defs/unmatched, and does not say that, when absent, each blank takes `incorrect` if it has an accept pattern and `manual` otherwise (fill-in.md footnote 6, line 47-50).

Impact: Readers may assume a single question-level default.

Suggested fix: Add: if absent, each blank takes its own default, `incorrect` with an accept pattern and `manual` without one.

Prioritized recommendations

P1: Define the fill-in stem: which blocks form it and the outcome of a `[^slug]` in each context. (Findings: F1, F2, F26, F33, F34)

P1: Define the empty blank and the pending fill-in response, and add a fill-in grading corpus. (Findings: F5, F11, F23)

P2: Map each fill-in strategy to a multiple choice strategy for choice blanks. (Findings: F4, F22)

P2: Move every parse error of blank definitions from the corpus into the Additional Rules table. (Findings: F3, F6, F7)

P2: Settle the accept block of a short answer blank. (Findings: F17, F18)

P3: Align the shared list, lazy line and exact literal grammars with CommonMark and with each other. (Findings: F37, F38, F39, F40, F41, F42)

P3: Apply the mechanical fixes: grammar copies, schema descriptions and the choice blank term. (Findings: F8, F9, F10, F19, F29, F30, F32)

GitHub issues

Each block below is a complete issue, ready to paste.

````
--- ISSUE 1 ---
# [Spec] Define the fill-in stem and where a blank may appear

**Labels:** high, medium, low

## Problem

The spec does not say which paragraphs form the stem, and allows two behaviors for a `[^slug]`
in the wrong place. Options: (a) the stem is the last paragraph before the definitions; a
`[^slug]` in an earlier paragraph is an error with a clear message. (b) The stem is every block
 from the first paragraph with a blank up to the definitions; blanks may appear in several
paragraphs. Recommended: (b), since cloze texts often span paragraphs. In both cases, give one
outcome per context: code (fenced, indented, code span) is literal text; a heading, list item,
table cell, blockquote, emphasis or link text of the stem gives `misplaced-blank`; the inline
parse of the whole paragraph comes first; `[^x](url)` and `[^x][ref]` are links.

## Evidence

- `docs/question-types/fill-in.md:155-160`: Prose allows two behaviors for a misplaced blank;
the rules table and the corpus require specific codes

  ```markdown
  illegal. A `[^slug]` found in any of them is not a blank: implementations MUST
  either reject the document or leave the text exactly as written, and MUST NOT
  silently drop it.
  ```
- `docs/question-types/fill-in.md:137-148`: The spec does not say which paragraphs form the
stem, so blanks in two paragraphs fail

  ```markdown
  stem  : inline_md? (blank inline_md?)+
  blank : "[^" SLUG "]"
  ```
- `docs/question-types/fill-in.md:155-157`: No example exercises blanks in list items, table
cells, blockquotes or a preamble paragraph

  ```markdown
  At the block level, the hosting block MUST be an ordinary paragraph. Blanks in
  ```
- `docs/question-types/fill-in.md:142`: `blank` matches the text of a link `[^x](url)`

  ```lark
  blank : "[^" SLUG "]"
  ```
- `docs/question-types/fill-in.md:168-172`: Split reading of `**start [^blank] end**` is legal
under the stem rule

  ```markdown
  interpreted as an unclosed bold element `**start `, followed by a blank and then
  the text ` end**`. In both approaches, the result would be illegal according to
  the spec, so both parsing strategies are valid.
  ```

## Impact

- **Prose allows two behaviors for a misplaced blank; the rules table and the corpus require
specific codes:** Two conforming implementations can give different lint codes, or one accepts
a document the other rejects. The corpus fixes behavior that the spec does not state.
- **The spec does not say which paragraphs form the stem, so blanks in two paragraphs fail:**
Authors write cloze texts that span paragraphs. The result is an error that names the wrong
cause, and another implementation can accept the same document.
- **No example exercises blanks in list items, table cells, blockquotes or a preamble
paragraph:** These MUST rules are not checked by the shared tests.
- **`blank` matches the text of a link `[^x](url)`:** The grammar and the markdown parser
disagree on whether the stem has a blank, so an implementation that follows the grammar reports
 `undefined-blank` for a valid link.
- **Split reading of `**start [^blank] end**` is legal under the stem rule:** The text allows a
 parsing strategy that gives a different result from the corpus.

## Suggested fix
````

```
- **Prose allows two behaviors for a misplaced blank; the rules table and the corpus require
specific codes:** Give one outcome per context in a table: for example, fenced code, indented
code and code spans are literal text; a `[^slug]` in a heading, list item, table cell or
blockquote of the stem, or inside emphasis or link text, is `misplaced-blank`. Remove 'either
... or'.
- **The spec does not say which paragraphs form the stem, so blanks in two paragraphs fail:**
State the rule: either the stem is the last paragraph before the definitions (and a `[^slug]`
in an earlier paragraph gives a specific error), or the stem is every block from the first
paragraph with a blank to the definitions. Add an example to the corpus.
- **No example exercises blanks in list items, table cells, blockquotes or a preamble
paragraph:** Add valid or invalid examples for each block kind once the behavior is settled.
- **`blank` matches the text of a link `[^x](url)`:** Add the lookahead of `CHOICE_ID` to the
rule (`"]"` not followed by `(` or `[`), and say in the text that `[^x](url)` and `[^x][ref]`
are links.
- **Split reading of `**start [^blank] end**` is legal under the stem rule:** Require the
markdown inline parse of the whole paragraph first, and state that a blank inside any inline
element is `misplaced-blank`. Remove the claim that both strategies agree.

## Acceptance criteria

- [ ] The spec states which blocks form the stem
- [ ] Each context of a `[^slug]` has one outcome and one code
- [ ] The corpus has a case for a list item, a table cell, a blockquote and a multi-paragraph
stem

--- END ISSUE 1 ---
```

````
--- ISSUE 2 ---
# [Spec] State the parse rules of the blank definitions

**Labels:** medium

## Problem

State that the definitions form one sequence of blocks, that a stem references each id at most
once (or that repeated references share one input), and give a `parse-error` rule for each
malformed definition: blank line before the choice list, text on a choice tag line (with a hint
 to use `/short-answer`), unknown kind suffix, unit on a non-numeric tag, a second definition
of an id, and a list item right after short answer content.

## Evidence

- `docs/question-types/fill-in.md:208-210`: The spec does not say whether blank definitions
must be contiguous

  ```markdown
  Definitions may appear in any order, and that order need not match the order
  the stem references them.
  ```
- `docs/question-types/fill-in.md:185-187`: A blank referenced more than once in the stem has
no rule

  ```markdown
  Every blank referenced by the stem MUST be defined, and every definition MUST
  be referenced by the stem.
  ```
- `docs/question-types/fill-in.md:481-482, docs/question-types/fill-in.md:206, docs/question-
types/fill-in.md:226, docs/question-types/fill-in.md:302-304`: Several Markdown parse errors of
 blank definitions are only in the corpus

  ```markdown
  | blanks[]             | error    | parse-error        | a short answer blank defines each
block at most once[^5]           |
  ```

## Impact

- **The spec does not say whether blank definitions must be contiguous:** An author who puts an
 explanation between definitions gets an error that points at the stem, not at the definition.
- **A blank referenced more than once in the stem has no rule:** An implementation can render
two inputs bound to one key, render one, or reject the document.
- **Several Markdown parse errors of blank definitions are only in the corpus:** An
implementation that reads only the spec can accept these documents or report other codes.

## Suggested fix

- **The spec does not say whether blank definitions must be contiguous:** State that the
definitions form one sequence of blocks, separated only by blank lines, and that the first
other block starts the epilogue. Consider a dedicated lint for a definition tag in the
epilogue.
- **A blank referenced more than once in the stem has no rule:** State that each blank id
appears at most once in the stem, with an error code (for example `duplicate-blank-reference`),
 or state that repeated references share one input.
- **Several Markdown parse errors of blank definitions are only in the corpus:** State each
rule in the text with MUST and add a `parse-error` row per case: blank line before the choice
list, text on a choice tag line, unknown kind suffix, unit on a non-numeric tag, a second
definition of an id (any kind), and a list item right after short answer content.

## Acceptance criteria

- [ ] Each corpus case of a malformed definition has a row in the Additional Rules table
- [ ] The spec says whether a paragraph may separate two definitions
- [ ] The spec has a rule for a repeated `[^id]` in the stem

--- END ISSUE 2 ---
````

````
--- ISSUE 3 ---
# [Spec] Map the fill-in strategy to the choice blank strategy

**Labels:** high, medium

## Problem

A choice blank is graded 'under the strategy the fill-in question declares', but multiple
choice has no `no-mistakes` or `all-or-nothing`. Recommended: use the exam table (`partial`,
`no-mistakes`, `all-or-nothing` give multiple choice `partial`; `symmetric` gives `symmetric`),
 and say that the example column holds the `symmetric` blank scores.

## Evidence

- `docs/question-types/fill-in.md:407-410`: Under no-mistakes and all-or-nothing, the multiple
choice strategy for a choice blank is not stated

  ```markdown
  * A **choice blank** is graded like a
    [multiple choice](multiple-choice.md#grading) question, under the strategy
    the fill-in question declares.
  ```
- `docs/question-types/fill-in.md:438-439`: The example table claims one set of blank scores
for every strategy, but the wrong pick scores differ by strategy

  ```markdown
  The "blank scores" column lists what each blank scored on its own, in that
  order, and every strategy is computed from those three numbers.
  ```

## Impact

- **Under no-mistakes and all-or-nothing, the multiple choice strategy for a choice blank is
not stated:** An implementation can raise an error, or score the blank with `symmetric`, for a
fill-in under `no-mistakes`.
- **The example table claims one set of blank scores for every strategy, but the wrong pick
scores differ by strategy:** Readers can conclude that a choice blank scores -0.5 under
`partial` and that the clamp in `partial` is what removes it.

## Suggested fix

- **Under no-mistakes and all-or-nothing, the multiple choice strategy for a choice blank is
not stated:** Add a mapping: a choice blank is scored with multiple choice `partial` under
`partial`, `no-mistakes` and `all-or-nothing`, and with `symmetric` under `symmetric`, as in
the exam table at `docs/exam.md:346-351`.
- **The example table claims one set of blank scores for every strategy, but the wrong pick
scores differ by strategy:** Say that the column holds the `symmetric` blank scores and that
the wrong pick scores 0 under the other strategies, or show both values.

## Acceptance criteria

- [ ] fill-in.md maps each strategy to a multiple choice strategy
- [ ] The example table says which strategy the blank scores column uses

--- END ISSUE 3 ---
````

````
--- ISSUE 4 ---
# [Spec] Define the empty blank and the pending fill-in response

**Labels:** high, medium

## Problem

State in docs/responses.md that a missing key and `null` are an empty blank, that `{}` is a
response with every blank empty (not a skip), and that a value of the wrong type is malformed
by the rule of the blank type. State that the instructor scores each pending blank in [0, 1]
and the strategy combines the scores. Add examples/grading/fill-in.yaml.

## Evidence

- `docs/question-types/fill-in.md:403`: An empty blank in the response is not defined

  ```markdown
  A blank the student left empty scores 0 under every strategy.
  ```
- `docs/question-types/fill-in.md:357-360`: For a pending fill-in, the spec does not say what
score the instructor supplies

  ```markdown
  A response to
  the question is settled only when the response to every blank is settled; if
  a blank leaves its response to the instructor, the whole question is pending.
  ```
- `docs/question-types/fill-in.md:389`: No grading corpus for fill-in

  ```markdown
  ## Grading
  ```

## Impact

- **An empty blank in the response is not defined:** The strategies `no-mistakes` and `all-or-
nothing` treat an empty blank differently from a wrong one, so the score depends on a rule the
spec does not give.
- **For a pending fill-in, the spec does not say what score the instructor supplies:** An LMS
cannot tell which value to ask the instructor for, and `no-mistakes` gives different results
for a blank score of 0.5.
- **No grading corpus for fill-in:** mdq-py and mdq-js can diverge on fill-in scores and
feedback without any shared test failing.

## Suggested fix

- **An empty blank in the response is not defined:** In `docs/responses.md`, Fill in the
blanks: a missing key and a `null` value are an empty blank; `{}` is a response with every
blank empty (the question is not skipped); a value of the wrong type is a malformed response by
 the rule of the blank's type. Add response cases to the grading corpus.
- **For a pending fill-in, the spec does not say what score the instructor supplies:** State
that the instructor scores each pending blank, in [0, 1], and that the strategy then combines
the blank scores as usual.
- **No grading corpus for fill-in:** Add examples/grading/fill-in.yaml covering the table rows,
 empty blanks, pending blanks and feedback order.

## Acceptance criteria

- [ ] docs/responses.md defines an empty blank
- [ ] The pending rule says what the instructor scores
- [ ] examples/grading/fill-in.yaml covers the example table, empty blanks, pending blanks and
feedback order

--- END ISSUE 4 ---
````

````
--- ISSUE 5 ---
# [Spec] Settle the accept block of a short answer blank

**Labels:** medium

## Problem

A short answer question MUST have an accept block, but a blank can have only a reject block.
Options: (a) require `[^id/short-answer]` for every short answer blank; (b) allow a reject-only
 blank and say so. Recommended: (a), the same rule as the question. Also reword line 314-315:
without accept patterns, `unmatched` defaults to `manual`, so the instructor grades what no
reject pattern settles.

## Evidence

- `docs/question-types/fill-in.md:306-308`: A short answer blank with only a reject block is
not addressed, unlike a short answer question that MUST have an accept block

  ```markdown
  A short answer blank has the two blocks of a short answer question:
  ```
- `docs/question-types/fill-in.md:314-315`: A blank with an empty accept block does not always
leave its responses to the instructor

  ```markdown
  `[^slug/short-answer]` block has no pattern has no `accept` list, and the
  instructor grades its responses, see [Automation](#automation).
  ```

## Impact

- **A short answer blank with only a reject block is not addressed, unlike a short answer
question that MUST have an accept block:** One implementation may require the accept block by
inheritance, another may not.
- **A blank with an empty accept block does not always leave its responses to the instructor:**
 Readers may think reject patterns or `unmatched` have no effect on an empty accept block.

## Suggested fix

- **A short answer blank with only a reject block is not addressed, unlike a short answer
question that MUST have an accept block:** State whether the accept block is required for a
blank, and add an example either way.
- **A blank with an empty accept block does not always leave its responses to the instructor:**
 Say that the blank has no accept list and that `unmatched` defaults to `manual`, so the
instructor grades the responses that no reject pattern settles.

## Acceptance criteria

- [ ] fill-in.md says whether a reject-only blank is valid, with a corpus example
- [ ] Line 314-315 mentions reject patterns and `unmatched`

--- END ISSUE 5 ---
````

````
--- ISSUE 6 ---
# [Spec] Make the list grammars follow one convention

**Labels:** medium, low

## Problem

`pattern_item` includes the list marker while the choice `item` does not; two `value` rules
collide in a fill-in grammar; a change of bullet character is read as one list. Recommended:
drop the marker from `pattern_item` (grammar.md:73-75 already says the item grammar applies
after the marker), rename the numeric `value` to `numeric_value`, and state that a change of
bullet starts a new list.

## Evidence

- `docs/references/patterns.md:252`: `pattern_item` includes the list marker, `item` does not

  ```lark
  pattern_item : ws? LIST_MARK ws content nl (blank_line* observations)?
  ```
- `docs/question-types/fill-in.md:223-224, docs/question-types/fill-in.md:239-241`: Two `value`
 rules collide in a fill-in grammar

  ```markdown
  with the `value` rule of [multiple choice](multiple-choice.md#body), and
  ```
- `docs/references/patterns.md:252-254, docs/question-types/fill-in.md:292`: `pattern_list`
accepts a change of bullet character

  ```lark
  LIST_MARK    : /[*+-]/
  ```

## Impact

- **`pattern_item` includes the list marker, `item` does not:** The two list rules that fill-in
 reuses follow opposite conventions; a literal reading of either one rejects the examples of
the other.
- **Two `value` rules collide in a fill-in grammar:** The fill-in grammar cannot be assembled
from the referenced snippets without renaming a rule.
- **`pattern_list` accepts a change of bullet character:** Grammar and markdown parse disagree
on how many lists the block has; the second list may become epilogue content in an
implementation that follows the markdown AST.

## Suggested fix

- **`pattern_item` includes the list marker, `item` does not:** Remove `ws? LIST_MARK ws` from
`pattern_item` (the list marker belongs to the list, as for `item`), or add it to `item` and
change grammar.md:73-75. Pick one convention for both.
- **Two `value` rules collide in a fill-in grammar:** Rename one of the rules, for example
`numeric_value` in numeric.md or `choice_value` in the choice types.
- **`pattern_list` accepts a change of bullet character:** State that all items use the same
bullet character, or say that the grammar reads consecutive lists with different bullets as one
 list.

## Acceptance criteria

- [ ] `item` and `pattern_item` follow the same convention
- [ ] No two rules referenced by fill-in share a name
- [ ] The spec says what a change of bullet character does

--- END ISSUE 6 ---
````

````
--- ISSUE 7 ---
# [Spec] Align lazy lines and exact literals with CommonMark

**Labels:** medium, low

## Problem

`LAZY_START` ignores setext underlines, excludes some lines that CommonMark continues (indented
 `>`, empty list items, backtick lines with a backtick in the info string), and `exact` needs
text after the opening backticks on the first line. Recommended: follow CommonMark in each
case, and state that a tag line that becomes a setext heading is a `parse-error`.

## Evidence

- `docs/references/grammar.md:156`: `LAZY_START` ignores setext heading underlines

  ```lark
  LAZY_START    : /(?!>|!(?!\[)|[*+-][ \t]|1[.)][ \t]|#{1,6}([ \t]|(?=[\r\n]))|`{3}|~{3}|(*[
\t]*){3,}(?=[\r\n])|(-[ \t]*){3,}(?=[\r\n])|(_[ \t]*){3,}(?=[\r\n]))[^ \t\r\n]/
  ```
- `docs/references/grammar.md:156, docs/question-types/short-answer.md:156`: `LAZY_START`
rejects lines that CommonMark reads as paragraph continuation

  ```lark
  lazy_content : ws? LAZY_TEXT (nl ws? LAZY_TEXT)*
  ```
- `docs/question-types/short-answer.md:152`: `exact` rejects a code span that opens at the end
of a line

  ```lark
  exact        : FENCE EXACT_TEXT (nl ws? LAZY_EXACT_TEXT)* FENCE ws?
  ```

## Impact

- **`LAZY_START` ignores setext heading underlines:** An implementation that follows the
grammar and one that follows the markdown AST disagree on whether the definition exists.
- **`LAZY_START` rejects lines that CommonMark reads as paragraph continuation:** Contrived
inputs parse differently in the grammar and in CommonMark.
- **`exact` rejects a code span that opens at the end of a line:** Minimal; authors rarely
break a literal right after the fence.

## Suggested fix

- **`LAZY_START` ignores setext heading underlines:** Add the setext underline (`=+` or `-+`
with optional trailing ws) to the exclusions of `LAZY_START`, and say what happens to a tag
line that becomes a setext heading (for example a `parse-error`).
- **`LAZY_START` rejects lines that CommonMark reads as paragraph continuation:** Limit the
`>`/`!` exclusion to at most three columns of indentation, exclude list markers only when non-
blank text follows, and exclude a backtick fence only when its info string has no backtick.
- **`exact` rejects a code span that opens at the end of a line:** Make the text on the first
line optional (`EXACT_TEXT?`), or state that the opening run must be followed by text on the
same line.

## Acceptance criteria

- [ ] LAZY_START excludes setext underlines
- [ ] The counter-examples of F41 parse as continuation
- [ ] An exact literal can open at the end of a line

--- END ISSUE 7 ---
````

````
--- ISSUE 8 ---
# Mechanical fixes

**Labels:** mechanical, medium, low

## Problem

Each fix below follows from its finding and needs no design decision, so they can be applied
together in one pass and reviewed by reading the diff. Severity still applies: a mechanical fix
 can matter a lot.

### F19 (medium): The two grammar snippets of `patterns` disagree

`docs/question-types/fill-in.md:197`

```lark
patterns         : answer | nl pattern_list
```

**Suggested fix:** Replace line 197 with `patterns : content | nl lazy_content | nl
pattern_list`, as on line 291.

### F32 (medium): Stem rule allows only one inline element between blanks

`docs/question-types/fill-in.md:141`

```lark
stem  : inline_md? (blank inline_md?)+
```

**Suggested fix:** Use `inline_md*` in the rule, or say that `inline_md` is a sequence of
inline elements.

### F8 (low): The Blank schema says a bare tag infers its kind from what follows

`schema/fill-in.yaml:76-79`

```yaml
      required here even though the surface syntax may leave it implicit
      (`[^capital]:` infers its kind from the definition that follows,
      while `[^size/numeric]:` states it).
```

**Suggested fix:** Write: ``[^capital]:` is a choice blank, while `[^size/numeric]:` and
`[^city/short-answer]:` state their kind.'

### F9 (low): The ShortAnswerBlank description lists answer forms that no longer exist

`schema/fill-in.yaml:125-128, schema/fill-in.yaml:139-141`

```yaml
      exactly like a short-answer question. Carries the same answer
      machinery: a list of accepted responses, a regular expression, or
      `accept`/`reject` pattern lists that can attach feedback.
```

**Suggested fix:** Write: 'Carries the same answer machinery: `accept` and `reject` pattern
lists, which can attach feedback.' In `accept`, cite `[^id/short-answer]` (alias `[^id/short-
answer/accept]`).

### F10 (low): A choice blank has three names: choice blank, multiple choice blank and type
multiple-choice

`docs/question-types/fill-in.md:213-215`

```markdown
## Multiple choice blanks

Multiple-choice blanks are represented by a reference definition followed by a
```
````

````
**Suggested fix:** Use 'choice blank' in prose and headings, and state once that its JSON
`type` is `multiple-choice`.

### F29 (low): `grading` description omits the per-type entry of an exam grading map

`schema/fill-in.yaml:38-39`

```yaml
`inherit`, the default, takes the exam's `grading` value, and
```

**Suggested fix:** Mention the `fill-in` entry of a grading map.

### F30 (low): `unmatched` description drops the per-blank default

`schema/fill-in.yaml:56-58`

```yaml
Applies to every short answer blank of the question, with the
```

**Suggested fix:** Add: if absent, each blank takes its own default, `incorrect` with an accept
 pattern and `manual` without one.

## Acceptance criteria

- [ ] F19: The two grammar snippets of `patterns` disagree
- [ ] F32: Stem rule allows only one inline element between blanks
- [ ] F8: The Blank schema says a bare tag infers its kind from what follows
- [ ] F9: The ShortAnswerBlank description lists answer forms that no longer exist
- [ ] F10: A choice blank has three names: choice blank, multiple choice blank and type
multiple-choice
- [ ] F29: `grading` description omits the per-type entry of an exam grading map
- [ ] F30: `unmatched` description drops the per-blank default

--- END ISSUE 8 ---
````